

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Simulación Montecarlo (I)

Suma de dos dados: solución analítica

Supongamos que queremos **calcular** la probabilidad de que la **suma de dos dados** sea **mayor que 8** al lanzarlos.

Cada dado tiene valores posibles $\{1, 2, 3, 4, 5, 6\}$, todos equiprobables. Hay en total $6 \times 6 = 36$ combinaciones posibles.

Las sumas mayores que 8 son: 9, 10, 11 y 12.

suma	combinaciones	cant. combinaciones
9	(3, 6), (4, 5), (5, 4), (6, 3)	4
10	(4, 6), (5, 5), (6, 4)	3
11	(5, 6), (6, 5)	2
12	(6, 6)	1

Total de casos favorables: $4 + 3 + 2 + 1 = 10$

$$P(\text{suma} > 8) = \frac{10}{36} = \frac{5}{18} \approx 0,277777777778$$

Suma de dos dados: aproximación programática

Supongamos que queremos un programa para **estimar** la probabilidad de que la **suma de dos dados sea mayor que 8** al lanzarlos.

- ▶ Cada experimento: lanzar 2 dados de 6 caras.
- ▶ Registrar si la suma es mayor que 8.
- ▶ Repetir muchas veces y calcular proporción de éxitos.

```
import random

N = 10000
 exitos = 0

for i in range(N):
    dado1 = random.randint(1,6)
    dado2 = random.randint(1,6)
    if dado1 + dado2 > 8:
        exitos = exitos + 1

prob = exitos / N
print('Probabilidad estimada:', prob)
```

¿Qué es la simulación Montecarlo?

Cuando obtener resultados exactos es complicado, la **simulación Montecarlo** permite estimarlos a partir de experimentos aleatorios en contextos con **incertidumbre** o **variabilidad**.

- ▶ Repetimos muchas veces un experimento aleatorio.
- ▶ Registramos los resultados obtenidos.
- ▶ Calculamos promedios, probabilidades u otras medidas de interés.

Idea clave: cuando no podemos calcular una solución exacta, podemos **simular muchas veces** y observar el comportamiento del sistema.

Biblioteca random

Para hacer simulaciones, Python ofrece la biblioteca `random`, que permite generar valores aleatorios.

Algunas funciones útiles:

► `random.randint(a, b)`

Genera un número entero al azar entre `a` y `b` (*ambos inclusive*). Ejemplo:

```
random.randint(1, 6)    # un número entero entre 1 y 6
```

► `random.choice(lista)`

Devuelve un elemento elegido al azar de una lista. Ejemplo:

```
random.choice(['cara', 'ceca'])    # simula una moneda
```

► `random.random()`

Devuelve un número decimal entre 0.0 y 1.0. Ejemplo:

```
random.random()    # 0.3746, 0.9211, etc.
```

Reproducibilidad: `random.seed`

En las simulaciones aleatorias, los resultados cambian cada vez que se ejecuta el programa.

Si queremos que una simulación produzca **exactamente los mismos resultados** en distintas corridas, podemos fijar una **semilla** (*seed*) usando la función `random.seed(...)`.

Ejemplo:

```
import random

random.seed(42)           # fija la semilla

print(random.randint(1,6))
print(random.randint(1,6))
```

- ▶ Si ejecutás este programa varias veces, **siempre imprime los mismos dos números**.
- ▶ Si cambiás el valor de la semilla, los resultados cambian.

Esto permite reproducir experimentos y verificar resultados de simulaciones de forma controlada.

Ejercicio: cartas numeradas

Ejercicio 1.

Queremos estimar la probabilidad de que **al menos una de dos cartas sacadas de un mazo numerado del 1 al 10 sea mayor o igual a 8**.

Se pide escribir un programa en Python que:

1. Simule la extracción de **2 cartas con reemplazo** usando `random.randint(1,10)`.
2. Repita la simulación **10.000 veces**.
3. Calcule la proporción de veces que al menos una carta es ≥ 8 .
4. Muestre la probabilidad estimada por consola.

La probabilidad exacta es 0,51. ¿El programa obtuvo una buena aproximación?

Variables categóricas y simulación

En muchos experimentos aleatorios, las variables no son numéricas sino **categóricas**.

Una **variable categórica** puede tomar uno de varios **valores posibles**, que representan *categorías* o *etiquetas*, en lugar de números.

Ejemplos típicos:

- ▶ El medio de transporte elegido por una persona: 'auto', 'colectivo', 'bicicleta', 'subte'.
- ▶ La bebida pedida en un café: 'café', 'té', 'mate'.
- ▶ La estación del año elegida al azar: 'verano', 'otoño', 'invierno', 'primavera'.

En Python, la función `random.choice()` permite simular este tipo de variables seleccionando aleatoriamente un elemento de una lista de categorías posibles.

Ejemplo:

```
import random

transportes = ['auto' , ' colectivo' , 'bicicleta' , 'subte' ]
print(random.choice(transportes)) # Imprime una de las categorías posibles
```

Así como `random.randint()` genera valores enteros aleatorios, `random.choice()` se usa para variables categóricas.

Otro ejemplo: monedas

Queremos estimar la probabilidad de que **al menos una moneda muestre cara** al lanzar 4 monedas.

```
import random

def obtener_lanzamientos():
    res = []
    for i in range 4:
        lado = random.choice(['C','S'])
        res.append(lado)
    return res

# Cuerpo principal del programa.
N = 10000
 exitos = 0
for i in range(N):
    lanzamientos = obtener_lanzamientos()
    if 'C' in lanzamientos:
        exitos = exitos + 1

prob = exitos / N
print('Probabilidad estimada:', prob)
```

Simulación: pedidos en un restaurante

Ejercicio 2.

En un restaurante, cada cliente elige su comida al azar entre cuatro opciones: 'pizza', 'empanadas', 'ensalada' y 'milanesa'.

Se atienden 3 clientes (independientes) y queremos estimar, mediante simulación, la probabilidad de que al menos uno pida pizza.

Para ello:

1. Representar las opciones como una lista: `menu = ['pizza', 'empanadas', 'ensalada', 'milanesa']`.
2. Repetir el experimento 10000 veces.
3. En cada simulación, elegir 3 comidas al azar usando `random.choice(menu)`.
4. Contar cuántas veces aparece al menos una 'pizza'.
5. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de simulaciones con al menos una pizza}}{\text{número total de simulaciones}}$$

La probabilidad exacta es 0,578125. ¿El programa obtuvo una buena aproximación?

Variables continuas y random

En muchos experimentos aleatorios, las variables no son discretas ni categóricas, sino **continuas**: pueden tomar cualquier valor real dentro de un intervalo.

Por ejemplo:

- ▶ La proporción de clientes que compran un producto.
- ▶ El tiempo (en horas) que tarda en completarse un proceso.
- ▶ La posición exacta donde cae un punto al azar dentro de un área.

La función `random.random()` genera un número **real entre 0 y 1** (sin incluir el 1).

```
import random

x = random.random()
print(x)      # Ejemplo: 0.372864...
```

La función `random.uniform(a, b)` es la generalización de `random.random()`: genera un número real en cualquier intervalo $[a, b]$.

```
y = random.uniform(5, 10)
print(y)      # Ejemplo: 7.4832...
```

Simulación Montecarlo con variables continuas

Ejemplo:

Supongamos que el tiempo (en minutos) que tarda un en llegar un colectivo a una parada está uniformemente distribuido entre 0 y 35 minutos. Queremos estimar la probabilidad de que llegue en **menos de 15 minutos**.

Para ello podemos usar Montecarlo:

1. Simular n colectivos usando `random.uniform(0, 35)`.
2. Contar cuántos tienen tiempo de llegada menor a 15.
3. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de colectivos que llegaron en menos de 15 min}}{\text{número total de colectivos simulados}}$$

```
import random

N = 10000
contador = 0

for i in range(N):
    tiempo = random.uniform(0, 35)
    if tiempo < 15:
        contador = contador + 1

prob = contador / N
print('Probabilidad estimada: ' + str(prob))
```

Ejercicio Montecarlo: temperaturas

Ejercicio 3.

Supongamos que la temperatura diaria en una ciudad durante el verano varía uniformemente entre 20°C y 35°C. Queremos estimar la probabilidad de que un día haga **menos de 25°C**.

Para ello podemos usar Montecarlo:

1. Simular n días usando `random.uniform(20, 35)`.
2. Contar cuántos días tuvieron temperatura menor a 25°C.
3. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de días con temp menor a 25°C}}{\text{número total de días simulados}}$$

La probabilidad exacta es 0,33333 (periódico). ¿El programa obtuvo una buena aproximación?

Simulación: demora total de entrega

Ejercicio 4.

Una empresa realiza entregas a domicilio y queremos estimar la probabilidad de que una entrega tarde **40 minutos o más**.

El tiempo total se compone de tres partes aleatorias e independientes:

- ▶ Preparación: $X_1 \sim \text{Uniforme}(5, 15)$ (ya conocido)
- ▶ Transporte: $X_2 \sim \text{Normal}(20, 5)$
- ▶ Demora por tráfico: $X_3 \sim \text{Normal}(10, 3)$

En Python, se puede generar valores normales usando: `random.gauss(mu, sigma)`.

Para ello:

1. Repetir el experimento 10000 veces.
2. En cada simulación, generar:
 - ▶ `prep = random.uniform(5, 15)`
 - ▶ `trans = random.gauss(20, 5)`
 - ▶ `traf = random.gauss(10, 3)`y calcular `total = prep + trans + traf`.
3. Contar cuántas veces `total ≥ 40`.
4. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de simulaciones con total} \geq 40}{\text{número total de simulaciones}}$$

5. Calcular el **tiempo promedio** de entrega como la media de los totales simulados.

Repaso de la clase

- ▶ Montecarlo: **repetir experimentos aleatorios** y analizar resultados.
- ▶ Pasos básicos:
 1. Generar números aleatorios.
 2. Definir la regla de éxito .
 3. Contar éxitos y calcular proporción o promedio.
- ▶ Próximo paso: simular procesos con memoria y sistemas más complejos.

Con lo visto, ya pueden resolver hasta el ejercicio 3.2 de la guía de ejercicios.